SportFlow Engine

SportFlow Engine: Engenharia de dados para Análise de Futebol

Aluno: Lucas Petry

Matrícula RA: 22352404

Descrição do projeto

O projeto SportFlow Engine tem como objetivo construir uma stack completa de engenharia de dados voltada à ingestão, processamento, análise e consumo de dados esportivos, com foco em futebol, utilizando dados provenientes da API SportsMonks.

A solução é projetada seguindo os princípios de arquitetura moderna de dados, adotando o paradigma de Lakehouse com arquitetura medalhão (Bronze, Prata e Ouro), permitindo a evolução dos dados desde seu estado bruto até a geração de insights estratégicos de alto valor.

O projeto de dados contempla todas as etapas do ciclo de vida:

- Ingestão de dados históricos em batch via API
- Armazenamento em Data Lake (camada Bronze)
- Processamento e limpeza (camada Prata)
- Modelagem analítica (camada Ouro)
- Consumo via dashboards interativos

Objetivo

Centralizar, estruturar e automatizar dados esportivos de forma escalável e organizada, transformando dados brutos em informações confiáveis e prontas para análise, com foco na geração de métricas e indicadores esportivos

Stakeholders e usuários finais

- Analistas e cientistas de dados interessados no domínio de negócio esportivo que busquem o consumo desses dados de forma estruturada para possíveis aplicações
- Entusiastas do assunto que busquem insights estatísticos e visualizações estratégicas

Problema e justificativa

A crescente disponibilidade de dados esportivos abre espaço para análises mais profundas e estratégicas. No entanto, esses dados geralmente estão dispersos, em formatos não estruturados ou com baixa qualidade, e são gerados em certa velocidade devido à rapidez de partidas e eventos. Este projeto propõe resolver esse problema por meio de uma arquitetura completa e flexível de engenharia de dados, garantindo organização, qualidade e perfomance no consumo de dados.

Definição e classificação dos dados

Fonte de dados

Os dados utilizados neste projeto são provenientes da API SportsMonks Football API, uma plataforma especializada no fornecimento de dados esportivos estruturados relacionados ao futebol.

A API disponibiliza informações sobre:

- Competições e ligas
- Times
- Jogadores
- Partidas
- Estatísticas de jogos
- Eventos durantes as partidas

Esses dados são acessados via requisições HTTP e retornados no formato JSON, sendo posteriormente processados e armazenados no ambiente de dados do projeto.

Formato dos dados

Os dados extraídos apresentam características típicas de dados semi-estruturados como formato JSON com estruturas aninhadas e presença de listas para eventos e estatísticas

Exemplo de estruturas:

Uma partida contém:

- times envolvidos
- placar
- status
- data

Um jogador pode conter:

- informações pessoais
- estatísticas por partida
- histórico em diferentes times

Classificação dos dados

- Dados Brutos (Raw Data)

  Origem direta da API

  Sem tratamento

  Armazenados na camada Bronze (Data Lake - MinIO)

  Alta fidelidade à fonte

  Estrutura original mantida

  Reprocessamento posterior

- Dados Tratados

Dados limpos, normalizados, estruturados e tratados

Tipagem consistente

Armazenados na camada Prata (Data Warehouse - PostgreSQL)

- Dados Analíticos

  Dados agregados e modelados

  Prontos para consumo e ingestão nas visualizações de dados

  Armazenados na camada Ouro (Data Warehouse - PostgreSQL)

Detalhamento

Os dados utilizados no projeto são classificados como dados históricos em batch

- Justificativa:
  - A API fornece dados sob demanda, sem necessidade de processamento em tempo real
  - O objetivo do projeto é análise histórica e geração de métricas agregadas
  - A complexidade adicional de um pipeline em streaming não se justifica para os objetivos propostos

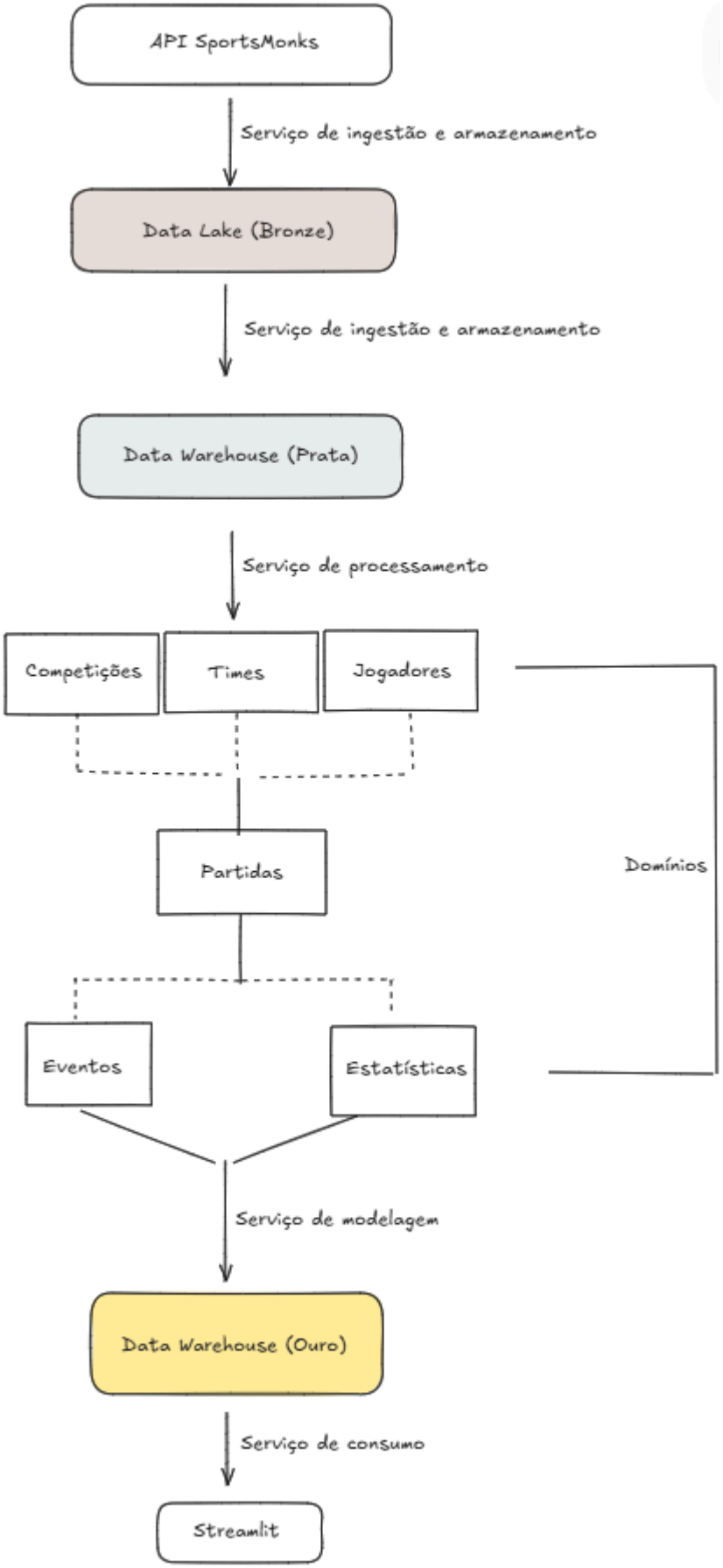
Frequência e volume estimado

As ingestões serão feitas de forma periódica com base nos calendários dos jogos, uma vez que ocorrendo as partidas, os dados precisarão ser atualizados conforme as mudanças de eventos e estatísticas.

Diante disso, os reprocessamentos necessários a cada atualização poderão ocorrer de forma incremental, apenas para os casos que forem atualizados pelas partidas e suas respectivas mudanças.

Em relação aos volumes de dados, entidades como competições e times, possuirão menor volume de armazenamento em razão de sua quantidade, pouco voláteis em razão do tempo. Em contrapartida, os jogadores, as partidas e seus respectivos eventos e estatísticas atualizam-se de forma mais recorrente apresentando uma quantidade de dados maior ao longo do tempo.

Domínios e Serviços



A definição de domínios e serviços no presente projeto segue princípios de arquitetura orientada a domínio (Domain-Oriented Data Architecture), com o objetivo de promover separação de responsabilidades, escalabilidade e organização lógica dos dados.

Os dados provenientes da API SportsMonks foram segmentados em domínios de negócio independentes, permitindo melhor governança, manutenção e evolução da plataforma de dados.

Cada domínio representa uma área semântica específica do contexto esportivo, enquanto os serviços são responsáveis pelo processamento e movimentação dos dados ao longo do pipeline.

Domínios:

- Competições:
    - Armazenar informações sobre as estruturas organizacionais do futebol
    - Dados de ligas, temporadas e países
    - Contextualizar as partidas dentro das competições específicas
    - Permitir analisar os jogos por campeonato e período
- Times:
    - Armazenar informações sobre os clubes e suas características
    - Dados sobre times, estádios e técnicos
    - Representar os participantes e permitir análises de desempenho coletivo
- Jogadores:
    - Armazenar dados individuais dos atletas
    - Dados de jogadores, posições e nacionalidades
    - Permitir análises e métricas individuais
- Partidas:
    - Domínio Central, registrando os principais eventos do jogo
    - Dados de partidas e rodadas
    - Permitir conectar times, competições e jogadores
- Estatísticas:
    - Armazenar os dados analíticos derivados das partidas
    - Dados analíticos das entidades dos domínios supracitados
    - Permitir fornecer métricas quantitativas e análises comparativas
- Eventos:
    - Armazenar informações de eventos granulares ocorridos durante as partidas
    - Dados de gols, cartões e substituições
    - Permitir análises mais detalhadas e temporais
    - Permitir representar acontecimentos em tempo de jogo

Serviços:

Os serviços definidos são responsáveis por operacionalizar o fluxo de dados entre os domínios e as camadas da arquitetura.

- Serviço de ingestão:

Responsável pela extração de dados da API e envio para a camada Bronze

Funções: Consumo da API SportsMonks, serialização dos dados em formato JSON/CSV e armazenamento do Data Lake

- Serviço de armazenamento:

  Responsável pelo armazenamento de dados brutos

  Funções: Persistência dos dados brutos, organização por domínio

- Serviço de processamento:

  Responsável pela transformação dos dados

  Funções: Limpeza, normalização, transformação de dados

- Serviço de modelagem:

  Responsável pela estruturação dos dados para consumo

  Funções: Criação das tabelas dimensionais, agregações e cálculos de métricas

- Serviço de orquestração:

  Responsável pelo controle, monitoramento e automação do pipeline

  Funções: agendamento de tarefas, monitoramento e controle de dependências

- Serviço de consumo:

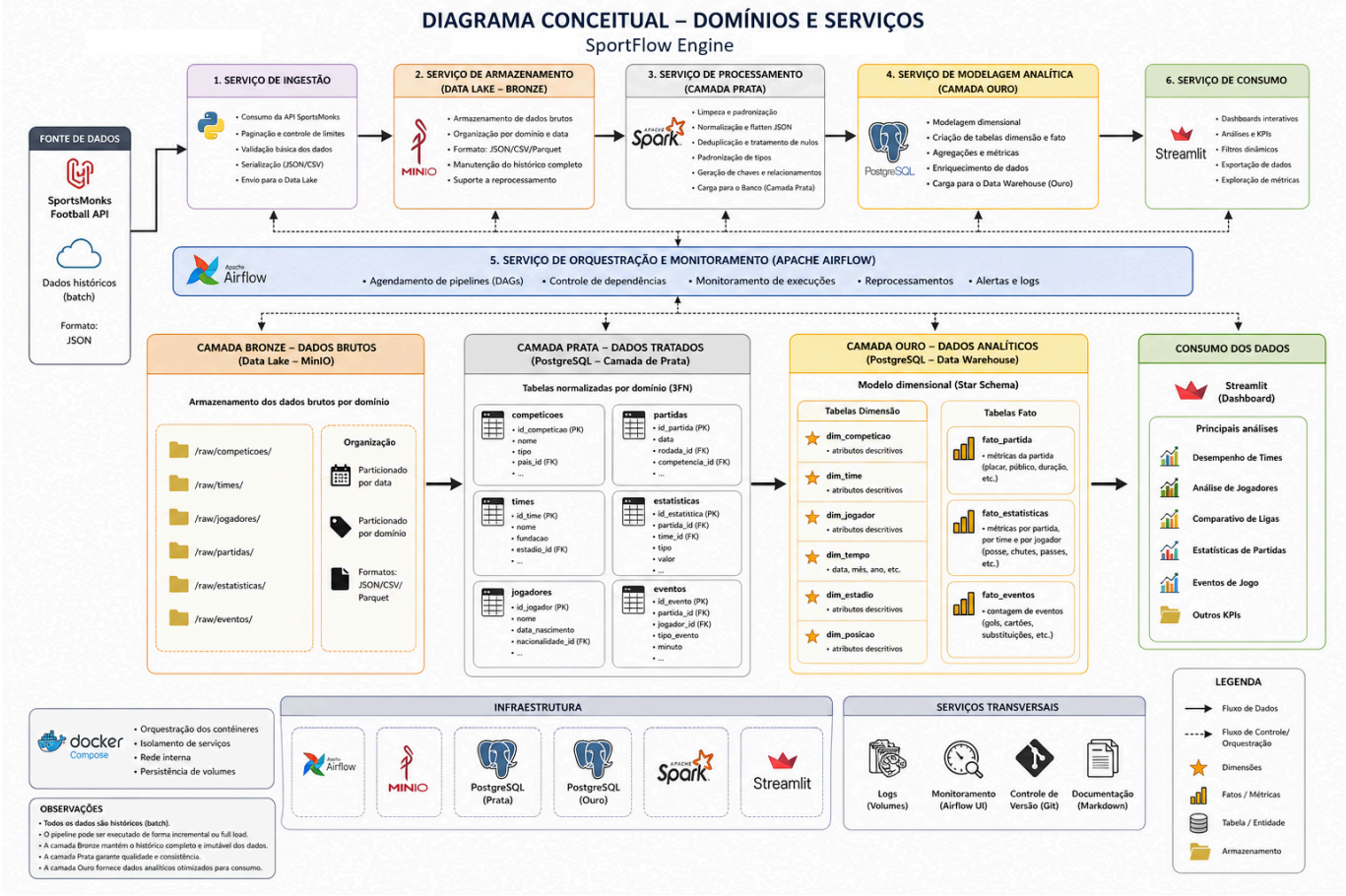
  Responsável pela disponibilização dos dados ao usuário final em dashboards com vizualizações estratégicas

  Funções: dashboards interativos, consultas e visualizações de métricas e indicadores

Os domínios são processados de forma independente na ingestão, mas passam a se relacionar nas camadas posteriores, especialmente:

Partidas conectam os Times e Competições Estatísticas e Eventos dependem de: Partidas, Jogadores e Times

Arquitetura | Fluxo de dados



A arquitetura proposta para o projeto é baseada no paradigma de Lakehouse, combinando as vantagens de Data Lakes e Data Warehouses em uma única solução integrada.

O modelo adotado segue a arquitetura medalhão, estruturando os dados em três camadas principais:

- Bronze (dados brutos)
- Prata (dados tratados)
- Ouro (dados analíticos)

Essa abordagem permite organizar o ciclo de vida dos dados de forma progressiva e eficiência no consumo.

Arquitetura Lakehouse

Justificativas:

- Armazenar ao mesmo tempo dados brutos em larga escala combinando com análise de forma estruturada
- Processamento e transformação de forma eficiente
- Flexibilidade para tratar com dados semi-estruturados
- **Separar as partes de armazenamento e processamento**
- Gerar a possibilidade de reprocessamento
- **Reduzir o acoplamento entre ingestão e consumo**

Arquitetura medalhão

Camada Bronze - Dados brutos (Data Lake)

- Tecnologia: MinIO

- Características:
  - Armazena dados **asis** diretamente da API
  - Formato em JSON/CSV
  - Dados imutáveis e organizados por domínio

Camada Prata - Dados Tratados (Data Warehouse)

- Tecnologia: PostgreSQL
- Características:
  - Dados limpos e normalizados (3FN)
  - Tratamento de inconsistências
  - Padronização e limpeza de dados
- Objetivo: garantir qualidade, integridade e consistência

Camada Ouro - Dados Analíticos de consumo

- Tecnologia: PostgreSQL
- Características:
  - Modelagem dimensional (**Star Schema**)
  - Tabelas fato e dimensão e dados agregados
- Objetivos: facilitar consumo e visualização nos dashboards, otimizar consultas

Fluxo de dados

O fluxo de dados segue uma arquitetura ETL, organizada nas seguintes camadas:

- Ingestão de dados
  - Consumo da API via python
  - Serialização dos dados
  - Saída: Dados **raw** enviados para a camada Bronze - MinIO
- Armazenamento
  - Dados armazenados no MinIO
  - Organização por domíno
  - Histórico completo
- Transformação/Processamento de dados
  - Via Apache Spark
  - Leitura, transformações e normalizações
  - Saída: Dados estruturados na camada Prata
- Modelagem
  - Criação de modelos dimensionais
  - Criação de métricas, indicadores e agregações
  - Saída: DW - Cama Ouro
- Orquestração

    - Via Apache Airflow
    - Execução de pipelines e DAG's
    - Monitoramento e reprocessamento

- Serviço/Consumo de dados

    - Vizualização estratégica com dashboards via Streamlit

Integração dos componentes

A arquitetura é **desacoplada** e **modular**

Ingestão **independente** do processamento Processamento **independente** do consumo Armazenamento centralizado

- Justificativas e vantagens: **escalabilidade**, manutenção e **reusabilidade** dos dados

Trade-offs e Decisões

- Batch vs Streaming

Menor complexidade e como os dados são históricos, a implementação em batch se torna mais adequada

- PostgreSQL como armazenamento nas camadas prata e ouro

Seu uso promove simplicidade, integração e boa performance para carga de dados

- Spark como forma de processamento

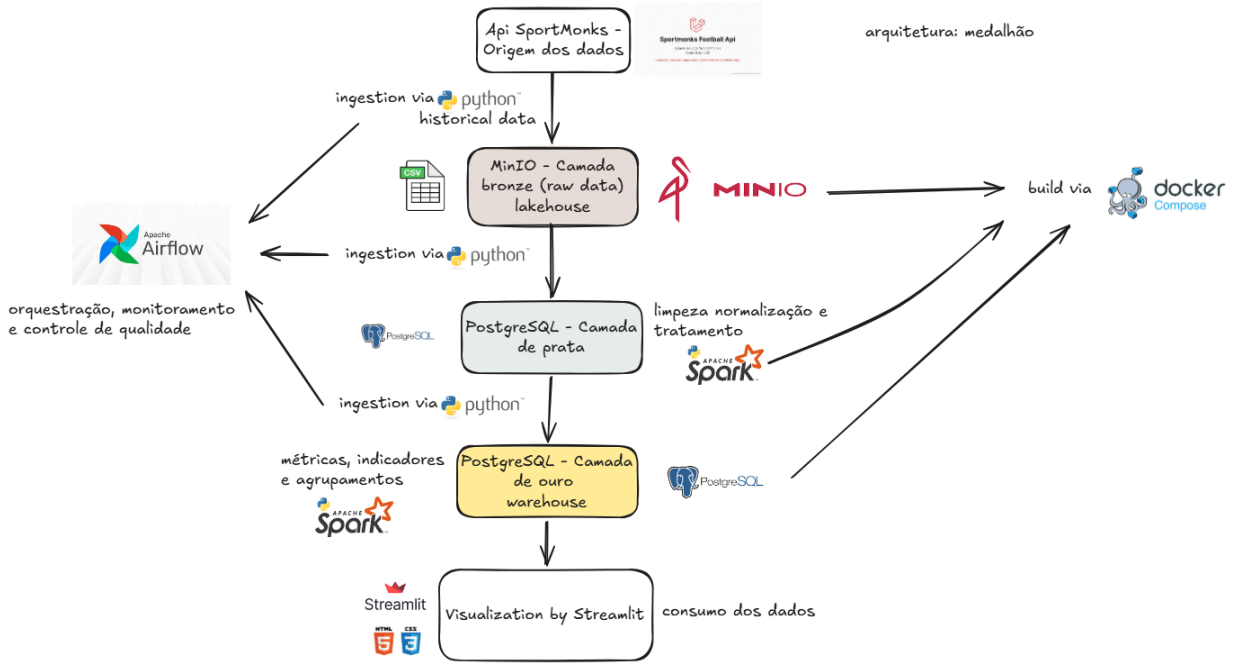
Excelente tecnologia quando considerado os tópicos de **processamento distribuído**, escalabilidade e os volumes de dados

- Data Lake com MinIO

Considerando a arquitetura e o fluxo de dados do projeto, torna-se viável em decorrência da compatibilidade com S3 e Docker

Dessa forma, a implementação da arquitetura permite passar por todo o ciclo de vida dos dados, separar de forma clara as responsabilidades, garantindo qualidade entre as etapas do fluxo.

Tecnologias



A escolha das tecnologias no projeto SportsLake Analytics foi orientada por critérios de simplicidade de implementação, escalabilidade, integração entre ferramentas, aderência ao paradigma Lakehouse e compatibilidade com ambientes containerizados

A stack foi definida de forma a cobrir todas as etapas do ciclo de vida dos dados, desde ingestão até consumo.

- Ingestão de dados
  - Realizada via scripts Python
  - Maior controle e facilidade de integração com API's
- Armazenamento
  - MinIO é utilizado como forma de armazenamento Lakehouse dos dados raw na camada Bronze
    - Compatibilidade com S3 e Docker
  - PostgreSQL é utilizado como forma de armazenamento Warehouse dos dados estruturados da camada Prata e dos dados analíticos da camada Ouro
    - Banco relacional, suportando modelagem normalizada e dimensional, boa performance para consultas analíticas visando consumo
- Processamento
  - Apache Spark é utilizado como tecnologia para processamento dos dados na camada de Prata
    - Permite processamento distribuído, grandes volumes de dados e integração com a arquitetura Lakehouse
- Serviço / Consumo
  - Streamlit será usado para construção das visualizações e dos dashboards
  - Fornece simplicidade, integração direta com o python e é open source, se destacando frente às ferramentas licenciadas

- Orquestração e Monitoramento
    - Airflow responsável pela orquestração das pipelines
    - Worflows com DAG's
    - Monitoramento e reprocessamento dos fluxos visando controle de falhas e observabilidade
- Infraestrutura
    - O ambiente será containerizado utilizando Docker Compose
    - Gerenciamento **simplificado e centralizado** dos serviços
    - **Isolamento e reprodutibilidade**

Considerações finais

O projeto foi concebido com o objetivo de implementar um pipeline completo de engenharia de dados, cobrindo todas as etapas do ciclo de vida dos dados, desde a ingestão até o consumo analítico.

A arquitetura proposta, baseada no paradigma Lakehouse e estruturada segundo o modelo medalhão (Bronze, Prata e Ouro), demonstrou ser adequada para organizar dados de forma progressiva, garantindo:

- Qualidade e consistência dos dados
- Separação clara de responsabilidades
- Facilidade de manutenção e reuso
- Eficiência no consumo analítico

Além disso, a utilização de tecnologias opensource contribui para a robustez da solução.

- Riscos e Limitações:

  Alguns riscos e limitações podem ser identificado e previstos, como:

    - Dependência de API externa
    - Crescimento do volume de dados ao longo da história
    - Possíveis dependências entre os serviços
    - Ausência de processamento em streaming
    - Infraestrutura local

Próximos passos

Diante de todo o planejamento e estruturação do projeto, visa-se a partir de então, começar as implementações de forma prática seguindo o fluxo de dados e as etapas de stack.

Organizar a estrutura dos diretórios do trabalho e começar o desenvolvimento dos códigos e scripts, além da montagem da infraestrutura será primordial para a parte 2.